IT305 Socket Programming Project

Fault-Tolerant Topic-Based File Distribution Service

Final Implementation and Result Analysis — DRAFT TEMPLATE

Merged Group: 95 & 120
Submission archive name: `FinalSubmission_Group_095_120.zip`

Group member	Roll number	One-line contribution (complete with the actual contribution before submission)
Dave Dhairya Kalpeshkumar	202401251	`[Fill in actual contribution]`
Bhatt Manthan Kamlesh	202401407	`[Fill in actual contribution]`
Dhruvam Panchal	202401415	`[Fill in actual contribution]`
Dave Aditya Sunil	202401413	`[Fill in actual contribution]`
Rudra Bhatt	202401461	`[Fill in actual contribution]`
Hari Sharma	202401195	`[Fill in actual contribution]`

> **Before final submission:** replace the marked contribution fields and all `TBD` performance entries with actual values from the three-computer lab experiments. The supplied performance measurements are not invented or prefilled.

1. Objective

Implement and evaluate a TCP socket-based topic file distribution service. The server maps each topic directory to a collection of files and transfers all files requested by a client. Part I compares single-threaded and multi-threaded service. Part II evaluates fault handling without recovery state, with checkpoint-based resume, and with enhanced parallel file downloads.

2. Implementation overview

2.1 Build layout

The submission contains `Part1/`, `Part2_Case1/`, `Part2_Case2/`, and `Part2_Case2_Enhanced/`. Each directory contains `server.c`, `client.c`, and `Makefile`; each Makefile creates the executable names `server` and `client`. Shared protocol and I/O code lives in `common/`; a synchronized copy sits inside every case folder so that each folder builds on its own, and `sync_common.sh --check` verifies the copies are identical.

Build each case from the submission root:

```
make -C Part1 clean all
make -C Part2_Case1 clean all
make -C Part2_Case2 clean all
make -C Part2_Case2_Enhanced clean all
```

The implementation targets Linux/POSIX because it uses `pthread`, POSIX sockets and filesystem calls, and Linux `sendfile()` for the enhanced sender.

2.2 Topic registry and deterministic order

At startup, the server scans the supplied `topics_root_dir`. Each immediate subdirectory becomes a topic. The server resolves absolute directory paths, records regular files and their sizes, sorts file names alphabetically, and builds a read-only open-addressed hash table for expected O(1) topic lookup. The registry is protected by a pthread read-write lock for concurrent lookups. Client-controlled topic and file names are validated as individual path components; the server selects files only from the startup registry.

2.3 Wire framing

Every message uses the design document's 8-byte header:

Field	Size	Encoding / meaning
Message type	1 byte	Opcode from `common/protocol.h`
Flags	1 byte	Bit `0x01` marks the final `FILE_DATA` chunk for a file
Status code	2 bytes	Big-endian application status; zero means success
Payload length	4 bytes	Big-endian byte count following the header

Payloads are read and written using `read`/`write`-equivalent loops over TCP, rather than assuming one call transfers the entire frame. Data chunks are at most 64 KiB. A frame (header plus payload) is handed to the kernel in a single `sendmsg()` call and all sockets use `TCP_NODELAY`;

sending the 8-byte header and the payload in two separate `send()` calls lets Nagle's algorithm hold the second small segment until the peer's delayed ACK, which stalls request/response exchanges. `FILE_HEADER` contains `uint64 file_size`, `uint64 start_offset`, `uint32 filename_len`, and the filename bytes. `CMD_RESUME` contains `uint32 topic_len`, `uint32 filename_len`, `uint64 byte_offset`, followed by topic and file name bytes.

Opcode	Value	Purpose
`CMD_GET`	`0x01`	Request a complete topic transfer
`CMD_RESUME`	`0x02`	Continue from a given file and byte offset
`FILE_HEADER`	`0x03`	Announce file size, start offset, and name
`FILE_DATA`	`0x04`	Carry one data chunk
`END_OF_TOPIC`	`0x05`	Complete a whole-topic transfer
`ACK`	`0x06`	Reserved by the protocol; currently unused
`CMD_MANIFEST`	`0x07`	Request the enhanced-case file manifest
`MANIFEST_DATA`	`0x08`	Return file names and sizes
`CMD_GET_FILE`	`0x09`	Request one file on a worker connection
`FILE_COMPLETE`	`0x0A`	Mark one enhanced file complete
`ERR_NOT_FOUND`	`0x10`	Topic or file not present
`ERR_BAD_OFFSET`	`0x11`	Resume offset invalid
`ERR_PROTOCOL`	`0x12`	Malformed or unsupported request

3. Part I — single-threaded and multi-threaded server

The required invocation is `./server <port> <topics_root_dir>`. The binary defaults to multi-threaded operation. For the controlled Part I comparison, an optional argument selects `single` or `multi`. The single-threaded server handles each accepted connection inline before accepting the next. The multi-threaded server creates a detached worker for each client; each worker parses requests and serves files independently.

The client invocation is `./client <server_ip> <server_port> <topic> <output_dir>`. It writes files into `output_dir/topic/`. The Part I client and server use `CMD_GET`, `FILE_HEADER`, `FILE_DATA`, and `END_OF_TOPIC`.

4. Part II — fault tolerance

4.1 Case 1: no session management

The server receives a failure probability from the command line. Before each new file and before each data chunk, it draws a pseudorandom value and may abruptly reset the connection using `SO_LINGER` with zero timeout. On a disconnect, the client reconnects, removes the partial topic output directory, and requests the topic again from byte zero. No checkpoint is used. This design provides the restart-from-zero baseline and is expected to retransmit data when a failure interrupts a transfer.

4.2 Case 2: checkpoint and resume

The client writes `.checkpoint.tmp` inside `output_dir/topic/`. It records the topic, current file, byte offset, and a completed-file counter as a fixed 512-byte record that is rewritten in place with one `pwrite()` after every received chunk. After a failure, it reconnects and sends `CMD_RESUME` with the last file and offset. When a checkpoint is loaded the stored offset is clamped to the length of the local file, so a stale or short record can never cause data to be skipped.

The server checks that the topic and file exist, and rejects offsets beyond the current file size. An offset equal to file size is treated as an already-completed file; the server proceeds to the next sorted file. The client leaves previously completed files in place. This follows the submitted design decision to keep progress on the client and validate each resume request on the server rather than preserving a server-side session table between connections. The Part I and Case 1 servers reject `CMD_RESUME`.

4.3 Case 2 — Enhanced: manifest and parallel files

The client first requests a manifest on a short-lived connection and parses all file names and sizes. Up to K workers (default K = 4; an optional trailing client argument can change it), each with its own persistent TCP connection, take whole files from a mutex-protected work queue. The per-file byte offset is kept in memory; after a dropped connection only that worker reconnects and resumes its file with `CMD_RESUME`, while the other workers continue. On disk, the client appends one line per completed file to an append-only journal (`.checkpoint.tmp`); progress inside an unfinished file is recovered from that file's length, because files are written strictly sequentially. Recording a completed file is therefore O(1) and no lock is shared between workers on the data path.

The server enlarges send and receive socket buffers to 256 KiB. For each `FILE_DATA` chunk it sends the 8-byte frame header with `MSG_MORE` (so the header leaves together with the payload) and then uses a bounded `sendfile()` for the payload. Bounding each sendfile operation at a 64-KiB chunk keeps fault-injection points aligned with the other cases. An optional server argument (`buffered`) switches to `pread()+send()` so the benefit of zero-copy can be measured separately from the benefit of parallel connections (vary K for the latter). File-level parallelism was selected instead of byte-range splitting because each topic contains many independent image files, avoiding out-of-order reassembly within one file.

5. Robustness and implementation notes

- Partial TCP reads and writes are handled by helper loops; incoming frame payloads are size-checked.
- File ordering is sorted for deterministic initial transfers and resumption.
- Empty files are represented by a `FILE_HEADER` and a zero-length final `FILE_DATA` frame.
- Client-supplied path components are rejected if empty, `.`/`..`, or containing path separators.
- `SIGINT` and `SIGTERM` stop the listener and the server then waits (up to about 10 s) for in-flight transfers to finish; server events and per-connection data-byte metrics are written to standard output/error so they can be redirected to files.
- Whole-topic replies end with `END_OF_TOPIC` and the server closes the connection; only the Enhanced per-file protocol keeps a connection open, so a stalled client cannot pin a worker (or the single-threaded server).
- Topic lookup is an exact hash match with a case-insensitive fallback.
- Fault injection draws from an independent per-connection `rand_r` seed (clock nanoseconds, pid and a connection counter mixed with splitmix64), so faults are independent across reconnects. Reusing `time ^ pid ^ thread-id ^ fd` as the seed gives only a handful of distinct seeds because file-descriptor numbers are reused.
- Client checkpoints and downloaded data are not `fsync()`ed per chunk: they survive a client process crash, not an operating-system crash. Per-chunk `fsync` made throughput depend on the disk rather than on the protocol.
- Clients reconnect immediately by default; `IT305_RECONNECT_DELAY_MS` adds an optional back-off (any back-off adds directly to completion time and must be reported if used).

6. Deviations from the submitted design document

1. Each per-case `server.c` and `client.c` is a small entry point into shared implementations under `common/`; this avoids duplicating framing, registry, and recovery code while still producing the required independent `server` and `client` binaries from each directory.
2. Case 2 retains the client-managed durable checkpoint and stateless per-request server validation described in Section 8.2 of the design. If the evaluator interprets "both client and server maintain some state" as requiring a persistent server-side session table, that interpretation should be confirmed with the instructor; a durable server session table is not implemented here.
3. The design document proposed SIGHUP-based dynamic directory reload and append-only centralized log files. The current implementation logs to stdout/stderr and loads the topic registry once at startup; dynamic SIGHUP reload and append-only file logging are not implemented. SIGINT/SIGTERM stop the listener and drain in-flight transfers before exit.
4. The enhanced workers reuse their TCP connection for multiple queued files; a worker reconnects if that socket fails. The manifest is fetched on a separate short-lived connection, so a transfer uses K data connections plus that one.
5. The Enhanced checkpoint (design section 4.2) is an append-only journal of completed files plus the on-disk length of unfinished files, instead of an array of per-file entries rewritten after every chunk; the rewrite was O(number of files) per chunk and serialised all workers behind one lock.
6. Case 2 checkpoint updates no longer use a temporary file, `rename()` and `fsync()` per chunk; see the durability note in section 5.
7. Added optional trailing server argument `sendfile|buffered` (Enhanced) so that the zero-copy enhancement can be isolated in the analysis.

7. Verification performed during development

The source is compiled with `-Wall -Wextra -Wpedantic` without warnings. `tools/verify.py` checks, for all four cases with six concurrent clients, that every downloaded file is byte-identical to the source with and without fault injection (including the 1-connection buffered Enhanced variant); that topics match case-insensitively and unknown topics fail cleanly; that the Part I and Case 1 servers refuse `CMD_RESUME`; that a client killed with `SIGKILL` in the middle of a transfer resumes from its checkpoint (Case 2 and Enhanced) and receives fewer bytes on the second run; and that `SIGINT` lets an in-flight transfer finish. The same suite was run under AddressSanitizer plus UndefinedBehaviorSanitizer, and the multi-threaded server and clients under ThreadSanitizer. AddressSanitizer found a one-byte stack-buffer overflow in checkpoint parsing (a 259-byte buffer for a `%259[^\n]` conversion), which is fixed.

These tests were run on a single development machine over the loopback interface. They validate protocol and recovery behaviour only; they are **not** the required three-computer performance results and must not be used as the report's measured lab results.

8. Performance analysis — replace all TBD values after measurement

Use actual runs on the assigned three-computer setup. Vary the number of clients for Part I. For Part II, vary failure probability for Case 1, Case 2, and Case 2 Enhanced. At least record elapsed completion time, server data bytes sent, client payload bytes received, useful payload bytes, redundant bytes, retries, and throughput.

Definitions:

- **Useful throughput (B/s)** = useful payload bytes successfully delivered / elapsed wall-clock seconds.

- **Retransmission overhead (%)** = 100 × redundant payload bytes / useful payload bytes.

8.1 Part I: aggregate throughput vs. client count

Concurrent clients	Single-threaded aggregate throughput (MB/s)	Multi-threaded aggregate throughput (MB/s)	Observations
1	TBD	TBD	TBD
2	TBD	TBD	TBD
4	TBD	TBD	TBD
Other tested values	TBD	TBD	TBD

Insert `Results/Part1_throughput.png` generated from the final measurements. Explain whether multi-threading improved aggregate throughput under the tested network and dataset, and identify any bottleneck.

8.2 Part II: measured metrics

Complete a separate table for each failure probability tested. The CSV should retain raw rows for each case, client count, and repetition.

Case	Failure probability	Client count	Wall time (s)	Server data bytes sent	Received payload bytes	Useful bytes	Redundant bytes	Retries	Aggregate throughput (MB/s)
Case 1	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Case 2	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Case 2 Enhanced	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Insert the final `Part2_Case1_throughput.png`, `Part2_Case1_overhead.png`, `Part2_Case2_throughput.png`, `Part2_Case2_overhead.png`, `Part2_Case2_Enhanced_throughput.png`, and `Part2_Case2_Enhanced_overhead.p` graphs. In the analysis, compare the overhead and completion-time impact of restarting from zero against resuming at a durable byte offset, then discuss whether file-level parallelism improved throughput in the three-computer environment.

9. Summary and conclusion — complete after measurement

The implementation supports the four required scenarios and provides client-side progress persistence for recovery. The expected qualitative distinction is that Case 1 loses partial progress after a dropped connection, whereas Case 2 resumes a file at its checkpoint. Case 2 Enhanced additionally overlaps downloads of independent files and uses bounded sendfile transmission. The final conclusion must be grounded in the collected measurements: report whether observed throughput, retransmission overhead, and completion time support those expectations, and mention any surprising results or limitations.